

Table des matières

Corrigé 08 — Données semi-structurées	1
Questions courtes	1
Questions pratiques	4
Diagnostics	7
Questions à développer	9
Mises en situation	10

Corrigé 08 — Données semi-structurées

Corrigé des exercices de `Exercices_08.md`. La numérotation est la même.

Questions courtes

1. Une donnée dont la structure est portée par la donnée elle-même plutôt que par le catalogue de la base, et qui peut donc changer d'une ligne à l'autre.

Théorie : `Theorie_08.md`, section 1.

2. Par exemple une API tierce, un flux d'événements et un webhook. Le point commun : vous ne décidez pas de la forme, elle appartient à celui qui l'émet et il la fait évoluer sans vous demander votre avis.

Théorie : `Theorie_08.md`, section 1.

3. Le `json` conserve le texte exact, l'ordre des clés et les doublons ; le `jsonb` normalise, dédoublonne et perd l'ordre. Le `json` s'écrit plus vite et se lit plus lentement, parce qu'il faut le ré-analyser à chaque accès. Seul le `jsonb` est indexable. On écrit `jsonb` par défaut.

Théorie : `Theorie_08.md`, section 2.

4. Quand la fidélité au texte reçu doit être prouvable, pour des raisons d'audit ou légales. Il faut alors pouvoir montrer ce qui est arrivé, octet pour octet.

Théorie : `Theorie_08.md`, section 2.

5. Un seul, `JSON`, qui accueille indifféremment un objet, un tableau ou une valeur simple. Il n'y a donc pas de choix à faire, là où PostgreSQL en impose un entre `json` et `jsonb`. Ce n'est pas une pauvreté : le compromis que PostgreSQL vous laisse arbitrer, la fidélité au texte reçu contre la vitesse de lecture, est tranché d'avance en faveur de la lecture, ce qui est le bon défaut pour un moteur analytique.

Théorie : Theorie_08.md, section 2.

6. Les deux rendent `{"a":1,"b":null}`, avec les deux clés : nos deux terrains sont d'accord. D'autres moteurs écartent la clé, dont Snowflake d'après sa documentation, où il faut écrire `OBJECT_CONSTRUCT_KEEP_NULL` pour la garder.

Des deux comportements, écarter la clé est le plus dangereux, et pour une raison qui n'est pas évidente : il rend deux documents structurellement différents à partir de deux lignes également remplies. Un test de présence de clé, `JSON_KEYS` ou un chemin comparé à `NULL`, répond alors « absent » pour une information qui a bien été traitée et qui valait simplement rien. La distinction entre « on ne sait pas » et « on n'a pas demandé » disparaît, et c'est exactement le problème du `NULL` du Cours 01, déplacé d'un cran.

Théorie : Theorie_08.md, section 3.

7. `->` rend du jsonb, `->>` rend du texte. On emploie la flèche simple tant qu'on continue de descendre dans le document, et la flèche double au dernier cran, quand on veut enfin la valeur.

Théorie : Theorie_08.md, section 4.

8. Parce que la flèche simple rend un morceau de document, donc "Canada" avec ses guillemets, et qu'aucune valeur ainsi écrite ne commence par Can. On écrit charge `->> 'pays' LIKE 'Can%'`, avec la flèche double, qui rend du texte nu.

Théorie : Theorie_08.md, sections 4 et 5.

9. Le `NOT IN` sur une colonne nullable. Le point commun est le mode de défaillance : la requête s'exécute sans erreur et renvoie zéro ligne, ce qui est une réponse plausible. Rien ne signale la faute.

Théorie : Theorie_08.md, section 5.

10. Non, jamais : la flèche double rend du texte, la flèche simple un morceau de document, des deux côtés. Sans conversion, le tri est alphabétique, donc 99 passe avant 1000. Le résultat a l'air trié et il est faux.

Théorie : Theorie_08.md, section 5.

11. On préfère la conversion tolérante dès que la source n'est pas maîtrisée, parce qu'une seule valeur invalide ferait tomber tout le lot avec un `CAST` sec. Ensuite, on compte les valeurs tombées à `NULL` : c'est ce compte qui dit si la donnée est saine ou s'il faut retourner voir le fournisseur. Elle s'écrit `TRY_CAST` sur l'entrepôt et `JSON_VALUE(doc, '$.chemin' RETURNING type)` sur PostgreSQL, cette seconde forme séparant en plus, par ses clauses `ON ERROR` et `ON EMPTY`, la valeur inconvertible de la clé absente.

Théorie : Theorie_08.md, section 5.

12. `jsonb_array_elements` sur PostgreSQL, `JSON_EACH` sur l'entrepôt. Toutes les deux doivent voir la colonne de la ligne courante, ce qu'une sous-requête ordinaire du `FROM` ne peut pas faire : c'est le `LATERAL` du Cours 07. PostgreSQL exige qu'on l'écrive, l'entrepôt s'en passe, et l'écrire des deux côtés reste

la bonne habitude. PostgreSQL a une seconde écriture, `JSON_TABLE`, celle de la norme, qui déclare le chemin du tableau et les colonnes voulues avec leur type ; l'entrepôt ne l'a pas.

Théorie : Theorie_08.md, section 6.

13. La colonne `key`. Sur un tableau, elle contient la position, comptée à partir de zéro, ce qui permet de conserver l'ordre d'origine : il compte dès que la position a un sens, et un tableau aplati perd son ordre sans elle. Sur un objet, la même colonne contient le nom de la clé, ce qui permet de parcourir un document dont on ne connaît pas les clés à l'avance.

Théorie : Theorie_08.md, section 6.

14. De deux façons. Avec `WITH ORDINALITY`, qui ajoute une colonne de position au résultat de la fonction. Ou avec la colonne `FOR ORDINALITY` d'un `JSON_TABLE`, qui la donne au même titre que les autres colonnes déclarées, et qui est la seule des deux à typer ces colonnes au passage, ce qui supprime les conversions une à une et l'oubli qui va avec.

Théorie : Theorie_08.md, section 6.

15. Il disparaît du résultat, sur les deux terrains : c'est mesuré, un document dont `lines` vaut `[]` ne ressort ni de `jsonb_array_elements` ni de `JSON_EACH`. L'outil qui le garde est le `LEFT` de `LEFT JOIN LATERAL`, vu au Cours 07 : les colonnes issues de l'aplatissement valent alors `NULL`. C'est une disparition silencieuse de plus, et elle est d'autant plus traître qu'un document sans ligne est souvent celui qu'on cherchait.

Théorie : Theorie_08.md, section 6.

16. `format = 'array'` donne le bon compte : il dit au moteur que le fichier est un tableau unique dont chaque élément doit devenir une ligne.

Des deux autres, la plus dangereuse est la lecture non structurée, qui rend une seule ligne contenant tout le fichier, sans un mot. La lecture ligne par ligne, elle, refuse le fichier en le nommant, `Malformed JSON in file ...`, at byte 2 in line 2, ce qui est un service : une erreur bruyante vaut mieux qu'un compte faux. Le symptôme de l'oubli est spectaculaire et facile à reconnaître quand on le connaît, trois lignes chargées au lieu de quarante mille, et c'est l'oubli le plus fréquent d'un premier chargement JSON. D'autres plateformes nomment la même option autrement, `STRIP_OUTER_ARRAY` sur Snowflake d'après sa documentation.

Théorie : Theorie_08.md, section 7.

17. Parce que le jour où le fournisseur ajoute un champ, la donnée brute l'a déjà et la colonne typée non. Typer au chargement perd définitivement ce qu'on n'avait pas prévu, et il faut redemander l'historique à la source.

Théorie : Theorie_08.md, section 7.

18. `@>` teste que le document contient le fragment donné. `?` teste la présence d'une clé. `?&` et `?|` testent la présence de toutes les clés d'une liste, ou d'au moins une.

Théorie : Theorie_08.md, section 8.

19. Parce que l'index GIN indexe la structure du document, pas le résultat d'une extraction. ->> produit une valeur calculée que l'index ne connaît pas. Deux parades : reformuler avec @>, ou créer un index sur l'expression elle-même. C'est un cas particulier de la sargabilité, que le Cours 10 généralise.

Théorie : Theorie_08.md, section 8.

20. La colonne fourre-tout où l'on met « le reste » ; le schéma qu'on refuse de décider en reportant la conversation avec le métier ; le champ stable rangé en JSON alors qu'il est présent dans tous les documents. La règle : le JSON est bon pour ce qui varie, mauvais pour ce qui est stable.

Théorie : Theorie_08.md, section 9.

Questions pratiques

P1. [OLTP]

```
SELECT jsonb_build_object(
    'client', h.customerid,
    'commandes', jsonb_agg(h.salesorderid ORDER BY h.salesorderid)
) AS document
FROM sales.salesorderheader AS h
GROUP BY h.customerid;
```

jsonb_agg accepte un ORDER BY interne, ce qui rend le document reproductible. Sans lui, l'ordre du tableau dépend du plan d'exécution et peut changer d'une exécution à l'autre, ce qui casse toute comparaison octet à octet.

Théorie : Theorie_08.md, section 3.

P2. [OLAP]

```
SELECT (c.charge ->> 'client')::INTEGER AS client,
    COUNT(DISTINCT c.charge ->> 'commande') AS nb_commandes,
    SUM((l.value ->> 'montant')::DECIMAL(19,4)) AS total
FROM LAB008_BRUT AS c,
    JSON_EACH(c.charge -> 'lignes') AS l
GROUP BY 1
ORDER BY total DESC;
```

Le COUNT(DISTINCT ...) est nécessaire parce que l'aplatissement a multiplié les lignes : une commande de trois articles apparaît trois fois. C'est l'erreur la plus fréquente après un aplatissement, et elle gonfle silencieusement les comptes. Le type de la somme n'est pas indifférent non plus : la conversion se fait en DECIMAL(19,4), celui de SalesAmount dans l'entrepôt, un type plus court changeant le total sans rien signaler.

Théorie : Theorie_08.md, sections 4 et 7.

P3. [OLTP]

```
-- Version fautive : tri alphabétique.
SELECT document ->> 'commande', document #>> '{lignes,0,prix}'
FROM   labo08.commande
ORDER BY document #>> '{lignes,0,prix}' DESC
LIMIT 10;

-- Version correcte.
SELECT document ->> 'commande', (document #>> '{lignes,0,prix}')::numeric
FROM   labo08.commande
ORDER BY (document #>> '{lignes,0,prix}')::numeric DESC
LIMIT 10;
```

Elles diffèrent parce que le premier tri compare des chaînes caractère par caractère : 99 passe avant 1000 puisque 9 vient après 1 dans l'ordre alphabétique.

Théorie : Theorie_08.md, sections 4 et 6.

P4. Sur l'entrepôt, les deux écritures :

```
-- En aplatissant : il faut une jointure externe pour que la ligne survive.
SELECT c.charge ->> 'commande' AS commande
FROM   LABO08_BRUT AS c
LEFT JOIN LATERAL JSON_EACH(c.charge -> 'lignes') AS l ON true
WHERE  l.value IS NULL;

-- Sans aplatir, plus direct.
SELECT charge ->> 'commande' AS commande
FROM   LABO08_BRUT
WHERE  (charge -> 'lignes') IS NULL
      OR JSON_ARRAY_LENGTH(charge -> 'lignes') = 0;
```

Notez les parenthèses autour de charge -> 'lignes' dans le IS NULL : sans elles, le moteur lit charge -> ('lignes' IS NULL) et répond No function matches ... json_extract(JSON, BOOLEAN). C'est le piège de priorité de la section 4, rencontré ici pour de vrai, et il est bruyant, donc sans gravité.

La seconde écriture est préférable, et je la garde : elle dit exactement ce qu'elle cherche, alors que la première demande au lecteur de comprendre pourquoi une jointure interne aurait fait disparaître précisément les lignes qu'on voulait. C'est le LEFT du Cours 07 dans un habit neuf, et le rappel que sans lui l'aplatissement perd ce qui est vide.

Théorie : Theorie_08.md, section 6.

P5. [OLTP]

```

CREATE INDEX idx_commande_doc ON labo08.commande USING GIN (document);
ANALYZE labo08.commande;

-- Emploie l'index.
EXPLAIN ANALYZE SELECT COUNT(*) FROM labo08.commande WHERE document @> '{"client": 29825}';

-- Ne l'emploie pas.
EXPLAIN ANALYZE SELECT COUNT(*) FROM labo08.commande WHERE document ->> 'client' = '29825';

```

Le premier plan montre un parcours d'index, le second un parcours séquentiel. Les deux requêtes renvoient pourtant le même compte.

Théorie : Theorie_08.md, section 8.

P6. [OLAP] On aplatit, puis on classe, puis on filtre le rang. QUALIFY fait tenir le tout en une requête.

```

SELECT c.charge ->> 'commande' AS commande,
       (l.value ->> 'produit')::INTEGER AS produit,
       (l.value ->> 'montant')::DECIMAL(19,4) AS montant
FROM   LAB008_BRUT AS c, JSON_EACH(c.charge -> 'lignes') AS l
QUALIFY ROW_NUMBER() OVER (PARTITION BY c.charge ->> 'commande'
                           ORDER BY (l.value ->> 'montant')::DECIMAL(19,4) DESC) = 1;

```

La conversion doit apparaître dans le ORDER BY de la fenêtre aussi, faute de quoi le classement serait alphabétique et le résultat faux. C'est le piège du module qui se cache dans une clause qu'on relit rarement.

Théorie : Theorie_08.md, section 6.

P7. [OLAP]

```

CREATE OR REPLACE TABLE brut AS
SELECT json AS charge
FROM   READ_JSON_OBJECTS('livraison.json', format = 'array');

CREATE OR REPLACE VIEW commande_typee AS
SELECT charge ->> 'commande' AS numero,
       (charge ->> 'client')::INTEGER AS client,
       TRY_CAST(charge ->> 'date' AS DATE) AS date_commande,
       charge AS document_brut
FROM   brut;

```

L'option format = 'array' est ce qui fait la différence entre une ligne et quarante mille, et TRY_CAST évite qu'une seule date mal écrite fasse tomber la vue entière.

La vue garde une référence au document brut. C'est ce qui permet d'ajouter une colonne le jour où le fournisseur ajoute un champ, sans rien recharger.

Théorie : Theorie_08.md, section 7.

P8. [OLTP]

```
SELECT SUM(JSON_VALUE(donnees, '$.montant' RETURNING numeric)) AS somme_valide,  
       COUNT(*) FILTER (WHERE JSON_EXISTS(donnees, '$.montant')  
                          AND JSON_VALUE(donnees, '$.montant' RETURNING numeric) IS NULL)  
                          AS valeurs_ecartees  
FROM   sinistre;
```

JSON_VALUE ... RETURNING numeric convertit et rend NULL au lieu de tomber, et SUM ignore les NULL : la somme est donc celle des montants valides, sans qu'on ait à filtrer en amont. Le second compte est aussi important que le premier, car une somme calculée sur des données partielles sans dire combien manquent est un chiffre sans qualité connue, et c'est là que JSON_EXISTS gagne sa place : il sépare « la clé est là mais la valeur ne se convertit pas », qu'il faut compter, de « la clé n'existe pas », qui n'est pas le même incident.

L'écriture d'avant la norme reste utile à connaître, parce qu'on la croise partout et qu'elle a un défaut discret :

```
SELECT SUM((donnees ->> 'montant')::numeric) FILTER (  
  WHERE donnees ->> 'montant' ~ '^-[0-9]+(\.[0-9]+)?$') AS somme_valide,  
       COUNT(*) FILTER (  
  WHERE donnees ->> 'montant' IS NOT NULL  
    AND donnees ->> 'montant' !~ '^-[0-9]+(\.[0-9]+)?$') AS valeurs_ecartees  
FROM   sinistre;
```

L'expression régulière fait office de TRY_CAST, et le FILTER du Cours 06 conditionne l'agrégat. Sur six documents dont un {"montant": "n/a"}, un {"autre": 1} et un {"montant": null}, les deux écritures rendent la même somme, 212,00, et deux comptes différents : 2 pour la première, 1 pour la seconde. L'écart est le document à valeur nulle, que donnees ->> 'montant' rend NULL et que la condition IS NOT NULL écarte du compte. La vieille écriture le confond donc avec une clé absente, ce qui est exactement le piège que le cours enseigne par ailleurs.

Théorie : Theorie_08.md, section 5.

Théorie : Theorie_08.md, section 5.

Diagnosics

E1. La flèche simple rend un morceau de document, et un texte y est écrit avec ses guillemets : "S043697". Le convertir en VARCHAR ne les enlève pas, la comparaison porte donc sur neuf caractères là où l'analyste en compare sept, et rien ne correspond. Il faut la flèche double, qui rend du texte nu : WHERE (charge ->> 'commande') = 'S043697', qui trouve bien sa ligne. Comme toujours dans ce module, la requête fautive ne signale rien, elle renvoie zéro.

La seconde partie de la question est la plus utile. La même écriture appliquée au client trouve ses trois lignes, parce que 11000 est un nombre et qu'un nombre s'écrit sans guillemets en JSON : la flèche simple et la flèche double y donnent le même texte. Le piège ne frappe donc que les chaînes, et voilà pourquoi il survit si bien aux essais. Quelqu'un qui vérifie son réflexe sur une colonne numérique en conclut que la flèche simple fait l'affaire, puis se casse le nez trois semaines plus tard sur une colonne de texte, sans faire le lien. La règle de la section 4 ne souffre donc pas d'exception à retenir : flèche simple tant qu'on descend, flèche double au dernier cran, quel que soit le type qu'on croit trouver au bout.

Théorie : Theorie_08.md, section 5.

E2. document ->> 'montant' rend du texte, donc le ORDER BY trie alphabétiquement. Les dix premières lignes sont celles dont le montant commence par le caractère le plus élevé, pas celles dont le montant est le plus grand.

```
SELECT document ->> 'commande', (document ->> 'montant')::numeric
FROM   commande
ORDER BY (document ->> 'montant')::numeric DESC
LIMIT 10;
```

Théorie : Theorie_08.md, section 5.

E3. Un aplatissement écrit ainsi se comporte comme une jointure interne : une commande dont le tableau lignes est vide ou absent disparaît, ce qui est mesuré sur les deux terrains. C'est le même défaut que le LATERAL sans LEFT du Cours 07, et il produit ici un rapport où les commandes vides, précisément celles qui méritent une investigation, sont invisibles.

```
SELECT c.charge ->> 'commande' AS commande, (l.value ->> 'produit')::INTEGER AS produit
FROM   commande_brute AS c
LEFT JOIN LATERAL JSON_EACH(c.charge -> 'lignes') AS l ON true;
```

Théorie : Theorie_08.md, section 6.

E4. Le fichier contenait un tableau, et l'option de forme manquait. Faute de la recevoir, la lecture non structurée a produit une seule ligne contenant tout le contenu du fichier, sans le moindre avertissement. Trois lignes veut dire trois fichiers lus. La correction tient en un mot : READ_JSON_OBJECTS('livraison.json', format = 'array').

```
COPY INTO brut FROM @stage FILE_FORMAT = (TYPE = JSON, STRIP_OUTER_ARRAY = TRUE);
```

Théorie : Theorie_08.md, section 7.

E5. ->> rend du **texte**, et jsonb_array_elements attend un jsonb. Il faut la flèche simple, puisqu'on continue de descendre dans le document.

```
SELECT jsonb_array_elements(document -> 'lignes') FROM commande;
```

C'est l'illustration la plus nette de la règle de la section 4 : la flèche double se réserve au dernier cran.

Théorie : Theorie_08.md, section 4.

E6. Quatre défauts, parmi lesquels une bonne réponse en cite au moins trois.

Le moteur ne peut rien garantir sur ces quatre informations : ni type, ni obligation, ni valeur par défaut, ni clé étrangère sur le mode de paiement. Un montant peut arriver en texte, une ligne peut n'avoir pas de statut, et rien ne le signalera.

Chaque lecture paie une extraction et une conversion, sur des colonnes lues en permanence. C'est le coût que la section 9 décrit comme repoussé sur tous les lecteurs suivants.

L'indexation devient un travail : il faut un index d'expression par champ, ou un index GIN qui ne sert pas les accès par flèche.

Enfin, la documentation disparaît. Le catalogue de la base ne dit plus qu'une commande a un statut, donc personne ne le sait sans lire du code applicatif. Comme les quatre champs sont présents depuis trois ans, ils ne sont plus du semi-structuré : ce sont des colonnes qui s'ignorent.

Théorie : Theorie_08.md, section 9.

Questions à développer

A1. Une bonne réponse défend la règle par la nature du contrat. Une colonne est un engagement vérifié par le moteur : le type est garanti, l'obligation est garantie, la relation est garantie. Un champ de document n'est rien de tout cela, c'est une convention que chaque lecteur doit connaître et respecter par lui-même. Payer ce prix a du sens quand la forme varie vraiment, parce qu'aucune colonne ne pourrait la suivre. Le payer pour une information stable, c'est renoncer aux garanties sans rien obtenir en échange.

Le motif attendu est celui de l'entrepôt : garder le document brut dans une couche d'accueil, pour l'audit et la reprise, et matérialiser en colonnes ce qui s'est stabilisé, dans une couche exploitée qui se régénère depuis la première. On tient ainsi les deux bouts, la fidélité et la performance, sans avoir à choisir.

Est également acceptable une réponse qui nuance la règle en soulignant qu'un champ « stable » ne l'est que jusqu'à preuve du contraire, et qui propose de surveiller la fréquence des clés pour décider quand promouvoir.

Théorie : Theorie_08.md, section 9.

A2. L'argument est réel et il faut commencer par le reconnaître : typer au chargement évite une extraction et une conversion à chaque lecture, ce qui est un gain mesurable sur une donnée lue mille fois pour une écriture.

Mais il confond deux choses. Rien n'oblige à choisir entre stocker brut et exposer typé : on fait les deux, en gardant le document et en matérialisant les colonnes qui servent. Ce que la proposition supprime vraiment, ce n'est pas la conversion, c'est l'information qu'on n'avait pas prévue. Le jour où un champ apparaît, la version typée-au-chargeement ne l'a jamais eu, et l'historique est perdu, sauf à redemander plusieurs années de données à un fournisseur qui n'a peut-être pas envie de les renvoyer.

Une bonne réponse conclut sur l'asymétrie des risques : l'un des deux choix coûte du calcul et se corrige, l'autre coûte de la donnée et ne se corrige pas.

Théorie : Theorie_08.md, sections 5 et 7.

A3. PostgreSQL traite le document comme un corps étranger. Le moteur ne sait rien de son contenu, donc il faut lui construire un index GIN, choisir les opérateurs qui l'emploient, et accepter qu'il soit gros et coûteux à maintenir. L'entrepôt n'offre pas d'index de ce genre, et n'en a pas l'usage : le Cours 04 a montré qu'un index n'y change ni le plan ni le temps, même sur des colonnes ordinaires, parce que la vitesse y vient du stockage en colonnes et non d'un chemin d'accès. Le document, lui, y reste un texte relu à chaque accès, une trentaine de fois plus lent qu'une colonne, mesuré. Les grands entrepôts vont plus loin en automatisant une partie du travail, Snowflake extrayant à l'écriture les éléments en colonnes d'après sa documentation, ce qui rend la lecture d'un élément extrait aussi rapide qu'une colonne ordinaire, sans rien demander.

La parade qui vaut sur tous les moteurs est la même, et c'est la conclusion : ce qu'on relit sans cesse, on cesse de l'extraire et on le matérialise. L'automatisme des grands entrepôts ne dispense pas d'y venir, puisque sa documentation le borne à 200 éléments par partition et en écarte tout élément qui porte un `null` ou des types mêlés. Sur un chemin lu en permanence, il reste préférable de matérialiser explicitement une colonne dans une table dérivée. Et surtout, aucune des deux optimisations ne répare une mauvaise décision de modélisation : un champ stable rangé en JSON reste une faute des deux côtés.

Théorie : Theorie_08.md, sections 8 et 10.

A4. Trois pratiques défendables.

Un test de non-régression sur les comptes : après chaque aplatissement, vérifier que le nombre de lignes produites égale la somme des tailles de tableau. Un écart signale un aplatissement sans jointure externe, ou un chemin mal orthographié.

Un contrôle de conversion : compter systématiquement les valeurs tombées à `NULL` après un `TRY_CAST`, et faire échouer le traitement au delà d'un seuil. C'est ce qui transforme une défaillance silencieuse en alerte.

Une revue de code qui interdit les comparaisons sans conversion explicite, éventuellement outillée par une recherche de motif dans le dépôt : tout : suivi d'une comparaison sans : : est suspect.

Celle à mettre en place en premier est la seconde, parce qu'elle attrape la faute la plus coûteuse, celle qui produit des chiffres faux plutôt que des lignes manquantes. Une réponse qui choisit la première est également recevable si elle argumente sur la fréquence plutôt que sur la gravité.

Théorie : Theorie_08.md, sections 4, 5 et 6.

Mises en situation

M1. Réponse défendable.

On stocke le message entier, tel qu'arrivé, dans une colonne unique d'une table d'accueil, avec l'horodatage de réception et le nom du fichier ou de la partition d'origine. Rien n'est typé à ce stade, et rien n'est rejeté.

On expose ensuite une vue qui type les cinq champs présents dans toutes les versions : identifiant, horodatage, conducteur, passager, prix. Les champs variables restent dans le document, accessibles par extraction quand on en a besoin. Les péages et les étapes, qui sont des tableaux, s'aplatissent à la demande.

Le jour où la version 4 ajoute un champ, il arrive dans la table d'accueil sans que rien ne change, puisque personne ne déclare de schéma. Exposer le nouveau champ revient à ajouter une colonne à la vue, et l'historique complet est disponible depuis le premier message reçu. C'est exactement l'argument de la section 7.

Une bonne réponse ajoute un point que l'énoncé suggère : avec trois versions en production, la vue doit tolérer l'absence d'un champ plutôt que de la traiter comme une anomalie, donc TRY_CAST et non CAST.

Théorie : Theorie_08.md, sections 7 et 9.

M2. La démarche attendue.

Comprendre. On inventorie les clés et leur fréquence. Sur l'entrepôt, JSON_EACH appliqué à un objet plutôt qu'à un tableau rend le nom de chaque clé dans sa colonne key, ce qui donne directement le compte :

```
SELECT f.key AS cle, COUNT(*) AS occurrences,
       COUNT(*) * 100.0 / (SELECT COUNT(*) FROM sinistre) AS taux
FROM   sinistre AS s, JSON_EACH(s.complement) AS f
GROUP BY 1 ORDER BY 2 DESC;
```

Sur PostgreSQL, jsonb_object_keys joue le même rôle. C'est la requête que l'énoncé demande, et c'est la première chose à écrire face à une colonne dont personne ne sait le contenu.

Mesurer. On identifie quelles clés les rapports lisent réellement, et combien coûte chaque lecture. Un plan d'exécution suffit à montrer si le temps part dans l'extraction ou ailleurs.

Corriger. Le critère de promotion d'une clé en colonne combine deux choses : sa fréquence, une clé présente dans presque toutes les lignes n'est plus du semi-structuré, et son usage, une clé lue par les rapports mérite d'être matérialisée même si elle est moins fréquente. Une clé rare et jamais lue reste dans le document.

Ce qu'il ne faut pas faire, et c'est la partie qui distingue une bonne réponse : ne pas éclater toute la colonne en quarante colonnes. On perdrait la souplesse pour laquelle elle existait, beaucoup seraient vides, et on aurait remplacé un problème par un autre. Ne pas non plus supprimer la colonne brute après extraction : elle reste la source de vérité et la seule façon de rattraper une erreur d'extraction.

Théorie : Theorie_08.md, section 9.

M3. Réponse défendable.

Les deux exigences ne s'opposent que si l'on cherche un seul stockage. La conception attendue en prévoit deux, avec une relation claire entre eux.

Le brut : les fichiers reçus, conservés tels quels, sur un stockage objet, ces services du cloud qui gardent des fichiers entiers désignés par une clé, avec leur nom, leur horodatage de réception et une empreinte. C'est ce qui satisfait l'audit à sept ans, et c'est la seule forme qui permette de reproduire un message à l'octet près. Une table d'accueil à colonne JSON unique référence chaque message et son fichier d'origine.

L'exploité : une table typée, alimentée depuis l'accueil, portant horodatage, marché et prix en colonnes, plus les métadonnées laissées en document puisque leur forme a changé trois fois. C'est elle que le tableau de bord interroge.

La garantie de régénération tient à une règle : la table exploitée ne reçoit jamais d'écriture directe, elle se reconstruit intégralement depuis l'accueil par une transformation versionnée. Si la transformation est fausse, on la corrige et on rejoue. Rien n'est perdu, puisque rien n'a été supprimé.

La contrainte qui interdirait jsonb est celle de l'audit à l'octet près. Le jsonb normalise : il dédouble les clés, en perd l'ordre et efface la mise en forme. Il ne permet donc pas de prouver ce qui a été reçu. C'est le cas rare, annoncé à la section 2, où l'on écrit json, ou plus raisonnablement où l'on conserve le fichier d'origine à côté et où la base ne sert qu'à l'exploiter.

Théorie : Theorie_08.md, sections 7 et 9.